

ADVANCE PROBLEM PROGRAMMING

SLO 2: Compare Paradigms

1. Compare Object-Oriented and Functional Paradigms Using Java

Feature	Object-Oriented Paradigm	Functional Paradigm
Basic unit	Objects and classes	Functions
Focus	Data and behavior	Computation and functions
Data	Objects can have mutable state	Prefer immutable data
Programming style	Uses classes, objects, inheritance, polymorphism	Uses lambda expressions and functional interfaces
Side effects	Commonly allowed	Usually minimized
Java features	Classes, objects, inheritance, interfaces	Lambda expressions, streams, functional interfaces
Example	Student class with methods	Lambda expression to process data

Object-Oriented Example:

```
class Calculator {  
    int square(int n) {  
        return n * n;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Calculator c = new Calculator();  
        System.out.println(c.square(5));  
    }  
}
```

```
}
```

Functional Example:

```
import java.util.function.Function;
```

```
public class Main {  
    public static void main(String[] args) {  
        Function<Integer, Integer> square = n -> n * n;  
  
        System.out.println(square.apply(5));  
    }  
}
```

Both programs produce:

25

The first program uses an object and method, while the second treats the calculation as a function.

2. Demonstrate How a Problem Is Solved Differently in Procedural and Object-Oriented Java Code

Problem: Calculate the area of a rectangle.

Procedural Approach

The program uses functions and passes data as parameters.

```
public class Main {  
  
    static double area(double length, double width) {  
        return length * width;  
    }  
}
```

```
public static void main(String[] args) {  
    double length = 10;  
    double width = 5;  
  
    System.out.println("Area = " + area(length, width));  
}  
}
```

Output:

Area = 50.0

Object-Oriented Approach

The data and the method that operates on the data are combined inside a class.

```
class Rectangle {  
    double length;  
    double width;  
  
    Rectangle(double length, double width) {  
        this.length = length;  
        this.width = width;  
    }  
  
    double area() {  
        return length * width;  
    }  
}
```

```

public class Main {
    public static void main(String[] args) {
        Rectangle r = new Rectangle(10, 5);

        System.out.println("Area = " + r.area());
    }
}

```

Output:

Area = 50.0

Difference: In the procedural approach, data and functions are separate. In the object-oriented approach, data and related behavior are combined into an object.

3. Strengths and Weaknesses of Declarative vs Imperative Paradigms

Aspect	Declarative Paradigm	Imperative Paradigm
Focus	Describes what should be done	Describes how it should be done
Strength	Simple and concise	Provides detailed control
Readability	Usually easier to read	Can become complex for large programs
Control	Less control over execution	More control over execution
Development	Faster for suitable problems	Requires more implementation details
Maintenance	Often easier to maintain	Can require more effort
Example	SQL	Java loops and statements

Strengths of Declarative Programming:

1. Simple and concise code.
2. Focuses on the required result.
3. Easier to understand for suitable problems.
4. Reduces unnecessary implementation details.

Weaknesses of Declarative Programming:

1. Provides less control over execution.
2. May not be suitable for every type of problem.
3. Performance optimization can sometimes be difficult.

Strengths of Imperative Programming:

1. Provides detailed control over program execution.
2. Suitable for algorithmic and step-by-step problems.
3. Makes memory and execution flow easier to control.
4. Widely supported by programming languages such as Java.

Weaknesses of Imperative Programming:

1. Programs can become lengthy and complex.
2. More implementation details must be specified.
3. Large programs can be harder to maintain and understand.